

JavaScript

Fundamentos

Da estrutura e do estilo prontos das últimas aulas ao primeiro código que pensa: variáveis, tipos, operadores, decisões e repetições.

DISCIPLINA

Programação — Web Coding (CC/P2)

MINISTRADO POR

Prof. Esp. Daniel Costa

script.js

```
const nome = "Web Coding";  
let nota = 8.5;  
  
if (nota >= 6) {  
  console.log(`${nome}: aprovado!`);  
}
```

O que vamos construir em Web Coding

Seis blocos, na ordem em que a própria Web é montada. Hoje avançamos do bloco 03 para o bloco 04.

01

Web

Como a Internet e as aplicações Web funcionam.

02

HTML5

A estrutura. O esqueleto de qualquer página.

03

CSS

A aparência. Cores, layout, responsividade.

04

JavaScript

O comportamento. Lógica de programação — hoje.

05

Bootstrap

Agilidade. Componentes prontos e responsivos.

06

JSON + SQLite

Os dados. Armazenar, consultar, persistir.

Hoje: estamos no bloco 04 — JavaScript. Ainda sem tocar na página (isso vem na Aula 5): hoje é sobre aprender a **linguagem** em si, no console do navegador.

De onde partimos hoje

HTML deu a estrutura, CSS deu a aparência. Falta a terceira peça: o comportamento.

O QUE JÁ TEMOS

Página pronta e bonita

- HTML5 semântico (Aula 2).
- CSS com layout e responsividade (Aula 3).
- Zero lógica até agora: a página não "pensa".

O QUE MUDA HOJE

A linguagem que dá vida à Web

- Variáveis, tipos de dados, operadores.
- Decisões (condicionais) e repetições.
- Tudo isso, por enquanto, direto no console do navegador.

Manipular a própria página com JavaScript (DOM, eventos) é conteúdo da Aula 6 — hoje é 100% sobre lógica de programação.

Onde o JavaScript roda

JavaScript nasceu para rodar dentro do navegador — sem instalar nada além do que você já tem.

```
index.html

<body>
  ...
  <script src="script.js"></script>
</body>
```

01 O navegador lê o HTML de cima para baixo.

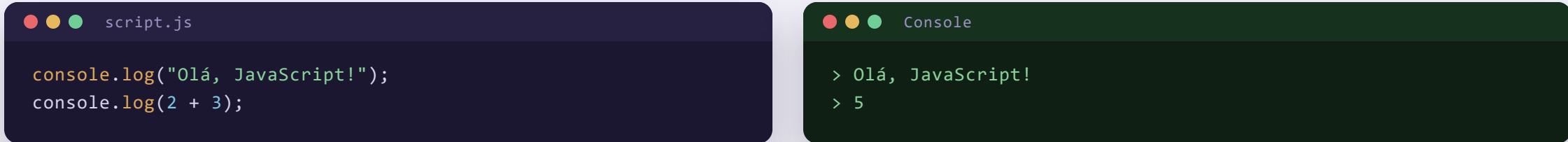
02 Ao encontrar a tag **script**, ele carrega e executa o arquivo `.js`.

03 Por isso o script fica no fim do **body**: a página carrega primeiro.

Assim como o CSS, o JavaScript pode ser interno (`<script>...</script>`) ou externo (arquivo `.js` ligado). Externo é o recomendado.

O Console do navegador

Antes de manipular a página, usamos o Console para ver os resultados do nosso código.



```
script.js  
  
console.log("Olá, JavaScript!");  
console.log(2 + 3);
```

```
Console  
  
> Olá, JavaScript!  
> 5
```

Abra o console com **F12** (ou botão direito → Inspecionar → aba Console). `console.log()` é o comando mais usado em toda a aula de hoje.

Guardando valores com let

Uma variável é um espaço nomeado na memória, onde guardamos um valor que pode mudar.

```
variaveis.js

let nome = "Ana";
let idade = 20;

idade = 21;
console.log(nome, idade);
```

declarar `let nome = valor;` cria a variável e já guarda um valor.

reatribuir Depois de criada, o valor pode mudar: `idade = 21;` (sem `let` de novo).

nomear Nomes descritivos, em *camelCase*: `nomeAluno`, não `x`.

Valores fixos com const

Quando um valor não deve mudar, usamos const em vez de let.

```
constantes.js  
  
const PI = 3.14;  
const NOME_ESCOLA = "Uninassau";  
  
PI = 4; // X erro: não é possível reatribuir
```

USE LET QUANDO

O valor muda

Contadores, resultados que se atualizam, entrada do usuário.

USE CONST QUANDO

O valor é fixo

Configurações, textos fixos, resultados de cálculo que não mudam depois.

Regra prática usada por profissionais: comece sempre com const; troque para let só se precisar reatribuir.

Os três tipos do dia a dia

Todo valor em JavaScript tem um tipo. Estes três aparecem o tempo todo.

tipos.js

```
let nome = "Ana";      // string
let idade = 20;        // number
let aprovado = true;   // boolean
```

TIPO	EXEMPLOS
string	Texto, sempre entre aspas: "Ana", 'Web Coding'.
number	Números, inteiros ou decimais: 20, 8.5.
boolean	Só existem dois valores: true ou false.

Ausência de valor e o operador typeof

Dois tipos especiais representam "nada" — e um operador revela o tipo de qualquer valor.

```
tipos2.js

let sobrenome;
console.log(sobrenome); // undefined

let resposta = null;

console.log(typeof 20);
console.log(typeof "Ana");
```

undefined Uma variável foi declarada, mas ainda não recebeu valor.

null Um valor "vazio" atribuído de propósito, pelo programador.

typeof Operador que devolve o tipo de qualquer valor, como texto.

Operadores aritméticos

As cinco operações matemáticas básicas da linguagem.

aritmeticos.js

```
console.log(10 + 3); // 13 - soma
console.log(10 - 3); // 7 - subtração
console.log(10 * 3); // 30 - multiplicação
console.log(10 / 3); // 3.33... - divisão
console.log(10 % 3); // 1 - resto da divisão
```

% (módulo) é o resto da divisão — muito usado para saber se um número é par ($n \% 2 === 0$).

Atribuição e incremento

Formas curtas de atualizar o valor de uma variável usando ela mesma.

```
atribuicao.js

let total = 10;
total += 5;    // total = total + 5
total -= 2;    // total = total - 2

let contador = 0;
contador++;    // contador = contador + 1
```

OPERADOR	EQUIVALE A
+=	x = x + valor
-=	x = x - valor
*=	x = x * valor
++ / --	Soma ou subtrai 1.

Comparação: == vs ===

Comparações sempre resultam em true ou false — a base de toda decisão no código.

```
comparacao.js

console.log(7 > 3);      // true
console.log(7 === "7"); // false
console.log(7 == "7");  // true
```

== Compara só o **valor**, convertendo o tipo se precisar. Evite.

=== Compara **valor e tipo**. Use sempre esta.

!== / != O oposto: "diferente de".

Regra de ouro em JavaScript: use sempre === e !==. O == gera comparações inesperadas e é fonte clássica de bugs.

Operadores lógicos

Combinam várias comparações em uma única decisão.

```
logicos.js

let nota = 8;
let presencas = 90;

console.log(nota >= 6 && presencas >= 75); // E
console.log(nota >= 9 || presencas === 100); // OU
console.log(!(nota >= 6)); // NÃO
```

OPERADOR	SIGNIFICA	É TRUE QUANDO
&&	E	Todas as condições são verdadeiras.
	OU	Pelo menos uma condição é verdadeira.
!	NÃO	Inverte o valor: true vira false.

Expressões: juntando tudo

Uma expressão é qualquer combinação de valores e operadores que produz um resultado.

```
expressoes.js

const nome = "Ana";
const nota1 = 7;
const nota2 = 9;

const media = (nota1 + nota2) / 2;

console.log(`${nome} tirou média ${media}`);
```

```
Console

> Ana tirou média 8
```

Template string (crases `` ``) permite inserir variáveis dentro do texto com `${variavel}` — sem precisar concatenar com `+`.

if / else: tomando decisões

O bloco só executa quando a condição entre parênteses é true.

```
condicional.js

const nota = 7;

if (nota >= 6) {
  console.log("Aprovado");
} else {
  console.log("Reprovado");
}
```

if Testa a condição. Se **true**, executa o bloco.

else Executa quando a condição do if é **false**. Opcional.

{ } As chaves delimitam o que pertence a cada bloco.

else if: mais de duas opções

Quando existem vários caminhos possíveis, encadeamos condições.

```
conceito.js

const nota = 8.5;
let conceito;

if (nota >= 9) {
  conceito = "A";
} else if (nota >= 7) {
  conceito = "B";
} else {
  conceito = "C";
}

console.log(conceito); // B
```

O JavaScript testa de cima para baixo e para no primeiro if/else if verdadeiro — a ordem das condições importa.

O laço for

Repete um bloco um número definido de vezes — ideal quando você sabe quantas repetições precisa.

```
for.js

for (let i = 1; i <= 5; i++) {
  console.log(`Volta número ${i}`);
}
```

```
Console

> Volta número 1
> Volta número 2
> Volta número 3
> Volta número 4
> Volta número 5
```

As três partes do for: **início** (let i = 1), **condição** (i <= 5) e **passo** (i++).

O laço while

Repete enquanto a condição for verdadeira — ideal quando não se sabe de antemão quantas vezes.

```
while.js

let senha = "";
let tentativas = 0;

while (senha !== "1234" && tentativas < 3) {
  tentativas++;
}
```

USE FOR QUANDO

Sabe o número de voltas

Percorrer uma lista de 1 a 5, repetir exatamente N vezes.

USE WHILE QUANDO

Depende de uma condição

Repetir "até acertar a senha" ou "enquanto houver saldo".

Cuidado com o **loop infinito**: se a condição nunca virar `false`, o código trava. Sempre garanta que algo dentro do laço muda a condição.

Antes do projeto: quatro desafios rápidos

Crie um arquivo `pratica.js` e resolva em sequência, conferindo cada resultado no console.

01 Crie duas **const** com suas notas e calcule a média em uma expressão.

02 Use **if/else** para exibir "Aprovado" ou "Reprovado" a partir da média.

03 Use um **for** para exibir os números de 1 a 10 no console.

04 Use **%** dentro de um **if** para dizer se um número é par ou ímpar.

Terminou os quatro? Essas técnicas são exatamente as peças do projeto de hoje — você já adiantou parte dele.

Sistema de Notas

Um script que calcula, decide e repete — reunindo tudo o que vimos hoje.

CONST/LET

Dados do aluno

Nome e notas guardados em variáveis e constantes.

EXPRESSÃO

Cálculo da média

Soma as notas e divide pela quantidade.

IF/ELSE

Situação final

Aprovado, em recuperação ou reprovado, conforme a média.

FOR

Turma inteira

Repete o cálculo para uma lista de vários alunos.

Tudo isso ainda roda só no **console** — a página HTML só reflete os resultados a partir da Aula 6.

script.js de partida

A estrutura já vem pronta — você conecta os conceitos de hoje para completá-la.

```
script.js

const alunos = [
  { nome: "Ana", notas: [7, 8] },
  { nome: "Bruno", notas: [5, 4] },
];

for (let i = 0; i < alunos.length; i++) {
  const aluno = alunos[i];
  const media = (aluno.notas[0] + aluno.notas[1]) / 2;

  if (media >= 6) {
    console.log(`${aluno.nome}: aprovado (${media})`);
  } else {
    console.log(`${aluno.nome}: reprovado (${media})`);
  }
}
```

Checklist de entrega

Antes de considerar pronto, confira se o seu script cobre todos os itens abaixo.

✓ Pelo menos uma const e um let usados corretamente

✓ Cálculo de média feito com uma expressão

✓ if/else decidindo a situação de cada aluno

✓ Um for percorrendo a lista de alunos

✓ Uso de console.log para mostrar cada resultado

✓ Nenhum erro no console ao rodar o script

Próxima aula, 21/09: funções, parâmetros, arrays e objetos — organizando essa mesma lógica em blocos reutilizáveis.

Sintaxe vista hoje

Uma colinha rápida com o vocabulário JavaScript desta aula.

SINTAXE	CATEGORIA	FUNÇÃO
<code>let / const</code>	Variáveis	Guardam valores; const não pode ser reatribuída
<code>string / number / boolean</code>	Tipos	Texto, número e verdadeiro/falso
<code>undefined / null / typeof</code>	Tipos	Ausência de valor e verificação de tipo
<code>+ - * / %</code>	Aritméticos	As operações matemáticas básicas
<code>+= -= ++ --</code>	Atribuição	Formas curtas de atualizar uma variável
<code>=== !== > <</code>	Comparação	Resultam sempre em true ou false
<code>&& !</code>	Lógicos	Combinam condições: E, OU, NÃO
<code>if / else / else if</code>	Condicionais	Executam código conforme uma condição
<code>for</code>	Repetição	Repete um número definido de vezes
<code>while</code>	Repetição	Repete enquanto uma condição for verdadeira

PARA LEVAR DAQUI

Quatro ideias que resolvem o resto

01

const é o padrão, let é a exceção

Só use let quando o valor realmente precisa mudar.

03

if/else é uma escolha, não uma sequência

Só um dos blocos executa — nunca os dois.

02

=== sempre, == nunca

Comparação com tipo evita bugs silenciosos.

04

Todo laço precisa de uma saída

Sem isso, for e while travam o navegador.

FIM DA AULA 4

Obrigado.

Hoje o código aprendeu a decidir e repetir. Na próxima aula, 21/09, organizamos essa lógica em **funções, arrays e objetos**.

DÚVIDAS SÃO BEM-VINDAS

Prof. Esp. Daniel Costa